

K-Nearest Neighbors: From First Principles

Why Do We Need KNN?

Some problems can be solved by looking at similar examples. If a new house is similar to nearby houses, its price may be similar. If a new customer behaves like customers who cancelled, they may also cancel. If a new flower has measurements similar to one species, it may belong to that species.

K-Nearest Neighbors, usually called KNN, is based on this simple idea: similar things are usually close to each other.

KNN does not learn coefficients like Linear Regression and does not build a tree like Decision Trees. Instead, it stores the training data and uses distance to make predictions when a new example arrives.

What Problem Does KNN Solve?

KNN can solve classification and regression problems.

For classification, KNN looks at the nearest training examples and chooses the most common class among them.

For regression, KNN looks at the nearest training examples and predicts the average of their target values.

The letter K means the number of neighbors used for prediction. If $K = 5$, the model looks at the 5 closest examples.

Distance-Based Learning

KNN depends on distance. Two points are close if their feature values are similar.

Imagine a map. If two houses are near each other on the map, they may share the same neighborhood. KNN uses the same idea, except the map may have many dimensions. Each feature becomes one dimension.

If a dataset has height and weight, each person can be placed on a two-dimensional graph. If a new person appears near a group of athletes, KNN may classify them as an athlete. If they appear near a group of non-athletes, KNN may classify them differently.

Euclidean Distance

Euclidean distance is the straight-line distance between two points.

For two points $A(x_1, y_1)$ and $B(x_2, y_2)$, the formula is:

$$Distance = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

For many features, the formula becomes:

$$Distance = \sqrt{\sum_{j=1}^d (x_j - a_j)^2}$$

Here, d is the number of features. The value x_j is feature j for the new point, and a_j is feature j for a training point.

Euclidean distance is common when features represent continuous numerical measurements.

Manhattan Distance

Manhattan distance measures distance by moving along feature axes rather than using a straight line.

For two dimensions, the formula is:

$$Distance = |x_2 - x_1| + |y_2 - y_1|$$

For many features:

$$Distance = \sum_{j=1}^d |x_j - a_j|$$

The name comes from moving through city blocks, where you cannot walk diagonally through buildings. Manhattan distance can be useful when movement along dimensions matters more than straight-line distance.

Choosing K

The value of K controls how local or smooth the prediction is.

If K is very small, such as $K = 1$, the model listens to only the nearest example. This can be very sensitive to noise.

If K is very large, the model listens to many examples. This can make predictions too smooth and may ignore local patterns.

A small K can overfit. A large K can underfit.

For binary classification, people often choose an odd K to reduce ties. If $K = 4$, two neighbors may vote for one class and two for another. If $K = 5$, ties are less likely.

Feature Scaling

Feature scaling is extremely important for KNN.

Suppose one feature is age, ranging from 0 to 100, and another feature is income, ranging from 0 to 200000. Income will dominate the distance calculation because its numbers are much larger.

KNN does not understand units. It only sees numbers. If features are not scaled, distance can become misleading.

Scaling puts features on comparable ranges so that no feature dominates only because of its unit.

How KNN Makes Decisions

When a new sample arrives, KNN calculates the distance from the new sample to every training example. Then it sorts the training examples by distance and chooses the K closest examples.

For classification, it uses majority voting among those neighbors.

Final Class = majority class among K nearest neighbors

For regression, it averages the target values.

$$\hat{y} = \frac{1}{K} \sum_{i=1}^K y_i$$

Here, y_i is the target value of neighbor i .

Worked Example

Suppose we classify a new fruit using sweetness and weight. The nearest 5 fruits are:

Neighbor	Class
1	Apple
2	Apple
3	Orange
4	Apple
5	Orange

Apple gets 3 votes. Orange gets 2 votes. KNN predicts Apple.

For regression, suppose the nearest 3 houses have prices 300000, 320000, and 310000.

$$\hat{y} = \frac{300000 + 320000 + 310000}{3} = 310000$$

So KNN predicts 310000.

Curse of Dimensionality

KNN becomes harder to use when there are many features. This problem is called the curse of dimensionality.

In high-dimensional space, points become far from each other. Distances become less meaningful because almost everything looks far away.

Imagine finding nearby houses on a simple street. That is easy. Now imagine comparing houses using hundreds of features. The idea of nearest becomes less clear.

This is why KNN usually works better with carefully chosen, scaled, meaningful features.

Computational Complexity

KNN has almost no training cost because it mainly stores the data.

Training cost is approximately:

$$O(1) \text{ or } O(n)$$

Prediction is slower because the model must compare the new sample with training examples.

For one prediction, the cost is roughly:

$$O(n \cdot d)$$

Here, n is the number of training examples and d is the number of features.

Memory cost is high because KNN stores the training data.

$$\text{Memory} = O(n \cdot d)$$

KNN is sometimes called lazy learning because most work happens during prediction, not training.

Advantages, Disadvantages, and Evaluation

KNN is simple, intuitive, and useful when similar examples have similar labels. It can handle classification and regression and does not assume a linear relationship.

Its disadvantages are slow prediction, high memory usage, sensitivity to feature scaling, sensitivity to irrelevant features, and poor performance in very high-dimensional data.

For classification, evaluate KNN using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC when appropriate. For regression, use MAE, MSE, RMSE, and R^2 .

Use KNN when the dataset is not too large, features are meaningful, and distance is a sensible way to compare examples. Avoid KNN when prediction speed is critical or the dataset has many noisy dimensions.